

# 第一章 概述

## 1.1 背景

随着高校校园规模的不断扩大，教学楼、宿舍楼、图书馆、体育馆、食堂等建筑布局日益复杂，新生入学、访客来访时常常面临“找路难”的问题。传统的纸质地图更新不及时、信息不全面，且无法提供动态导航服务；通用地图软件对校园内部道路、楼宇细节的覆盖不足，难以满足精细化导航需求。在此背景下，开发一款适配校园场景的导游系统，整合校园地理信息、提供精准路径规划与信息查询服务，成为解决校园导航痛点的迫切需求。

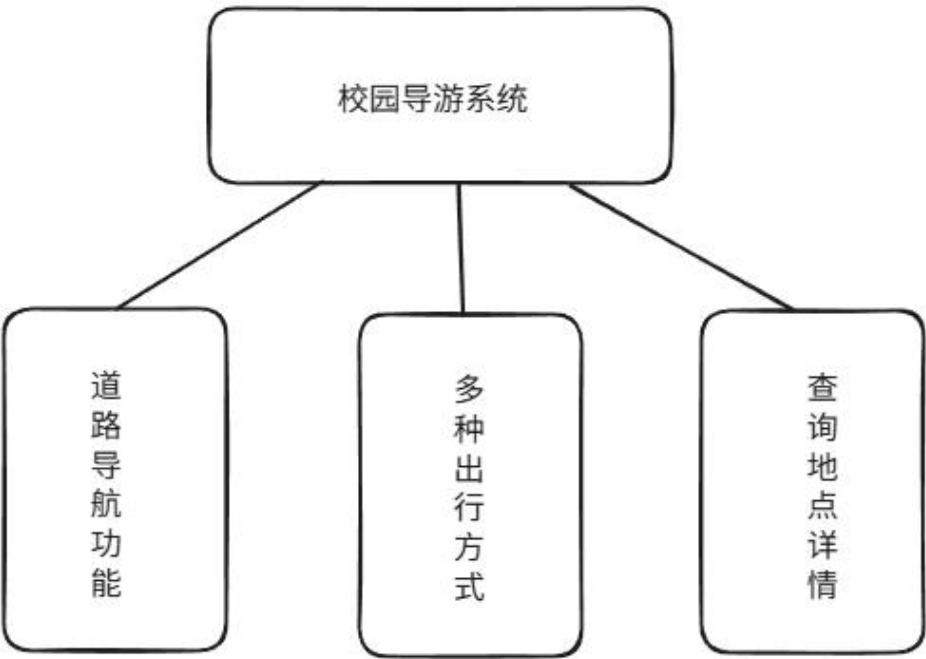


图 1-1 系统架构图

## 1.2 拟解决的问题

本校园导游系统旨在解决以下核心问题：

**校园景点与功能区快速查询：**支持用户通过名称检索目标地点，获取详细信息（位置、用途、开放时间等）。

**最优路径规划：**根据用户起点和终点及出行方式，提供最短距离和所需时间。

**校园信息可视化展示：**以图文结合的方式呈现校园布局，直观展示楼宇分布、道路连接关系。

## 1.3 价值

**实用价值：**解决校园导航痛点，为新生、访客提供精准路线规划（支持步行、自行车、电动车三种出行方式）、沿途建筑详情查询、开放时间实时展示等服务，缩短找路时间，提升校园出行效率与服务体验。

**教学价值：**作为数据结构与算法课程的实践载体，将图论（邻接表存储）、最短路径算法（Dijkstra）、地理信息计算（Haversine 公式）等理论知识转化为可落地的系统功能，强化“理论 - 实践”转化能力。

**拓展价值：**系统核心架构具备复用性，可迁移至公园、景区、工业园区等小型区域导航场景。

## 1.4 意义

**解决实际痛点：**针对校园导航的特殊性，提供定制化解决方案，弥补通用导航软件的不足。

**可扩展性意义：**系统的核心架构可迁移至公园、景区等小型区域导航场景，具备一定的复用价值。

## 第二章 相关工作

### 2.1 区域导航系统研究现状

从技术应用来看，现有校园导航系统多以路径规划为核心功能，主流实现方式是结合图论模型与最短路径算法。例如，部分系统采用 Floyd-Warshall 算法实现全节点间路径预计算，但该算法时间复杂度为  $O(n^3)$ ，在节点数量较多时存在性能瓶颈；另有系统采用 A\* 算法提升搜索效率，但需依赖精准的启发函数设计，在校园道路复杂场景下，启发函数的合理性较难保障。此外，现有系统在用户体验优化上存在明显短板，多数缺乏沿途信息展示、多出行方式适配、实时状态反馈等实用功能，难以满足用户多样化需求。

### 2.2 核心技术与算法应用现状

#### 2.2.1 地理信息可视化技术

地理信息可视化是区域导航系统的基础支撑，目前主流方案是集成成熟的第三方地图 API 或自建 GIS。百度地图、高德地图等开放 API 凭借稳定的接口服务、完善的可视化能力，成为轻量化系统的首选。本系统选用百度地图 API，正是基于其对经纬度定位的高精度支持、丰富的交互控件，以及对多终端设备的良好兼容性，能够快速实现校园地图的可视化呈现与路径轨迹绘制。

#### 2.2.2 路径规划算法

路径规划算法是导航系统的核心逻辑，现有研究中应用较广的包括 Dijkstra 算法、A\* 算法、Bellman-Ford 算法等。其中，Dijkstra 算法因实现逻辑简洁、稳定性高，成为静态最短路径计算的经典方案，尤其适用于节点间权重非负的场景（如校园道路距离）。相较于 A\* 算法，Dijkstra 算法无需设计复杂的启发函数，在校园节点数量适中（本系统 23 个核心节点）的场景下，能够以较低的开发成本实现精准的路径规划，因此成为本系统的算法选型。

---

同时，距离计算方法的选择直接影响路径规划的准确性。现有系统中，平面直线距离计算）适用于小范围、地形平坦的场景，但在跨区域或需要考虑地球曲率的场景下误差较大。Haversine 公式通过将经纬度坐标转换为地球表面两点间的球面距离，有效解决了欧氏距离的局限性，尤其适用于基于真实地理坐标的导航系统。本系统将 Dijkstra 算法的路径搜索与 Haversine 公式的距离计算相结合，既保证了路径搜索的最优性，又实现了距离计算的精准性，弥补了传统系统中“路径最短”与“实际距离最优”脱节的问题。

### 2.2.3 前后端开发技术

前后端分离架构已成为现代软件开发的主流模式，其核心优势在于前后端职责分离、开发效率提升、系统扩展性强。现有校园导航系统中，后端多采用 Spring Boot、Django 等框架实现数据接口与业务逻辑处理，前端则结合 Vue、React 等框架构建交互界面。本系统采用“Spring Boot 后端+原生 JavaScript 前端”的架构，既利用了 Spring Boot 在数据持久化（与 MySQL 对接）、接口开发上的高效性，又通过原生 JavaScript 降低了前端开发复杂度，实现了“轻量高效、易于维护”的开发目标，符合小型区域导航系统的技术选型需求。

## 2.3 现有系统不足与本系统创新点

通过对现有校园导航系统的调研分析，发现其普遍存在以下不足：一是路径规划功能单一，仅支持最短距离规划，缺乏多出行方式适配与沿途信息展示；二是校园信息查询不够便捷，未实现建筑开放时间的实时状态反馈与模糊搜索功能。

针对上述不足，本系统在相关工作基础上实现了三点创新：第一，在算法应用上，将 Dijkstra 算法与 Haversine 公式结合，同时适配步行、自行车、电动车三种出行方式，通过预设不同出行速度实现精准的时间预估；第二，在功能设计上，增加沿途建筑详情展示、开放时间实时状态判断（基于当前日期与时间）、节点模糊搜索等实用功能，提升用户体验；第三，在数据存储上，采用“MySQL 持久化存储+内存邻接表缓存”的方式，既保证了数据的安全性，又提升了算法遍历邻接节点的效率，为系统后续扩展动态数据更新功能奠定了基础。

# 第三章 算法设计

## 3.1 数据结构设计方案

系统核心数据结构包括 "节点" 和图的存储方法：

- 1. 节点 (NavNode)：代表校园内的关键地点，包含唯一标识 (id)、名称 (name)、经度 (lng)、纬度 (lat) 等属性，用于定位和始末位置的下拉框选项展示，图的存储 (NavEdge)。
- 2. 持久化存储：通过 MySQL 数据库，用 nav\_node 表和 nav\_edge 表分别存储节点和边数据，通过 NavEdgeMapper 等接口实现数据读写。

| id | name    | lng        | lat       | category | description | open_time_school | open_time_holiday |
|----|---------|------------|-----------|----------|-------------|------------------|-------------------|
| 1  | 南门      | 119.910106 | 28.463266 | 校门       | 学校南门，游客...  | (NULL)           | (NULL)            |
| 2  | 路口1     | 119.910077 | 28.465018 | (NULL)   | 三叉路口，右侧...  | (NULL)           | (NULL)            |
| 3  | 路口2     | 119.911639 | 28.464954 | (NULL)   | (NULL)      | (NULL)           | (NULL)            |
| 4  | 路口      | 119.910066 | 28.46599  | (NULL)   | (NULL)      | (NULL)           | (NULL)            |
| 5  | 教学楼11A栋 | 119.912225 | 28.464909 | 教学楼      | 阶梯教室，多...   | 6:30-22:30       | 6:30-21:30        |
| 6  | 学生事务中心  | 119.911028 | 28.464983 | (NULL)   | 4层楼，1楼：打... | 6:30-22:30       | 6:30-21:30        |
| 7  | 马克思主义学院 | 119.90914  | 28.464076 | 教学楼      | 5层楼：普通教...  | 6:30-22:30       | 6:30-21:30        |
| 8  | 教学楼6栋   | 119.908309 | 28.464189 | 教学楼      | 5层楼：普通教...  | 6:30-22:30       | 6:30-21:30        |
| 9  | 图书馆     | 119.907141 | 28.464343 | (NULL)   | 5层楼：普通教...  | 6:30-22:30       | 6:30-21:30        |
| 10 | 教学楼11B栋 | 119.912267 | 28.464454 | 教学楼      | 阶梯教室，多...   | 6:30-22:30       | 6:30-21:30        |
| 11 | 教学楼11C栋 | 119.912284 | 28.464109 | 教学楼      | 阶梯教室，多...   | 6:30-22:30       | 6:30-21:30        |
| 12 | 教学楼11D栋 | 119.912251 | 28.463808 | 教学楼      | 阶梯教室，多...   | 6:30-22:30       | 6:30-21:30        |
| 13 | 教学楼11E栋 | 119.912196 | 28.46355  | 教学楼      | 阶梯教室，多...   | 6:30-22:30       | 6:30-21:30        |
| 14 | 计算机中心大楼 | 119.912654 | 28.464912 | 教学楼      | 1楼为青训学院...  | 6:30-22:30       | 6:30-21:30        |
| 15 | 教学楼12A栋 | 119.912599 | 28.464731 | 教学楼      | 阶梯教室        | 6:30-22:30       | 6:30-21:30        |
| 16 | 教学楼12B栋 | 119.912621 | 28.464407 | 教学楼      | 普通教室        | 6:30-22:30       | 6:30-21:30        |
| 17 | 教学楼12C栋 | 119.912573 | 28.464048 | 教学楼      | 阶梯教室，多...   | 6:30-22:30       | 6:30-21:30        |
| 18 | 教学楼12D栋 | 119.912578 | 28.463734 | 教学楼      | 阶梯教室，多...   | 6:30-22:30       | 6:30-21:30        |
| 19 | 教学楼12E栋 | 119.912608 | 28.463482 | 教学楼      | 阶梯教室，多...   | 6:30-22:30       | 6:30-21:30        |
| 20 | 路口3     | 119.910087 | 28.465218 | (NULL)   | (NULL)      | (NULL)           | (NULL)            |
| 21 | 路口4     | 119.911644 | 28.465361 | (NULL)   | (NULL)      | (NULL)           | (NULL)            |
| 22 | 路口5     | 119.911647 | 28.465693 | (NULL)   | (NULL)      | (NULL)           | (NULL)            |
| 23 | 教学楼16栋  | 119.912246 | 28.465656 | 教学楼      | 5层楼，多为...   | 6:30-22:30       | 6:30-21:30        |

图 3-1 1

ENTITIES 

nav\_edge

id

from\_node

to\_node

distance

double

nav\_node

<

图 3-1 2

## 3.2 数据结构源代码

数据库表映射的 Java 类（存储地点详情和通过图设计道路线路的两张表）。

```
@Data 23 个用法
@TableName("nav_node")
public class NavNode {
    @TableId(type = IdType.AUTO)
    private Long id;
    private String name;
    private Double lng;
    private Double lat;
    private String category;
    private String description;
    private String openTimeSchool;
    private String openTimeHoliday;
}
```

图 3-2-1

```
@Data 8 个用法
@TableName("nav_edge")
public class NavEdge {
    @TableId(type = IdType.AUTO)
    private Long id;
    private Long fromNode;
    private Long toNode;
    private Double distance;
}
```

图 3-2-2

展示导航路线和用时：

通过枚举类型定义不同出行方式的速度，通过通过 Dijkstra 计算出“最短路径”，而最短路径包括多个节点，通过 calcDistance()方法传入最短路径对应的节点的列表，通过 Haversine 公式计算每个节点两点之间的距离累加求和即为最终的路径长度，后端根据前端的请求参数（起点、终点、出行方式）返回相应的距离和用时：

```

public enum TravelMode { 2个用法

    WALK( speed: 1.2),      // m/s 0个用法
    BIKE( speed: 4.0),      0个用法
    E_BIKE( speed: 6.0);    0个用法

    @Getter
    private final double speed;

    TravelMode(double speed) { this.speed = speed; }
}

```

图 3-2- 3

```

public List<NavNode> findPath(Long startId, Long endId) { 1个用法

    // 1. 查询所有节点和边
    List<NavNode> nodes = nodeMapper.selectList( queryWrapper: null);
    List<NavEdge> edges = edgeMapper.selectList( queryWrapper: null);
    // 2. 构建邻接表
    Map<Long, List<NavEdge>> graph = new HashMap<>();
    for (NavEdge e : edges) {
        graph.computeIfAbsent(e.getFromNode(), Long k -> new ArrayList<>()).add(e);
    }
    // 3. Dijkstra 初始化
    Map<Long, Double> dist = new HashMap<>();
    Map<Long, Long> prev = new HashMap<>();
    Set<Long> visited = new HashSet<>();
    for (NavNode n : nodes) {
        dist.put(n.getId(), Double.MAX_VALUE);
    }
    dist.put(startId, 0.0);
    // 4. 主循环
    while (true) {
        Long cur = null;
        double min = Double.MAX_VALUE;

        for (Long id : dist.keySet()) {
            if (!visited.contains(id) && dist.get(id) < min) {
                min = dist.get(id);
                cur = id;
            }
        }
    }
}

```

图 3-2- 4

```

// 计算路径总距离 (米)
public double calcDistance(List<NavNode> path) {
    double sum = 0;

    for (int i = 0; i < path.size() - 1; i++) {
        sum += haversine(
            path.get(i).getLat(),
            path.get(i).getLng(),
            path.get(i + 1).getLat(),
            path.get(i + 1).getLng()
        );
    }

    return sum;
}

// Haversine 公式 (地球两点距离)
private double haversine(double lat1, double lng1,
    double lat2, double lng2) {
    final double R = 6371000; // 地球半径 (米)
    double dLat = Math.toRadians(lat2 - lat1);
    double dLng = Math.toRadians(lng2 - lng1);
    double a = Math.sin(dLat / 2) * Math.sin(dLat / 2)
        + Math.cos(Math.toRadians(lat1))
        * Math.cos(Math.toRadians(lat2))
        * Math.sin(dLng / 2)
        * Math.sin(dLng / 2);

    double c = 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1 - a));
    return R * c;
}

```

图 3-2-5

展示不同出行方式的下拉框：

```

<div class="container">
    <div class="control-panel">
        <div class="form-group">
            <label for="mode">出行方式: </label>
            <select id="mode">
                <option value="WALK">步行</option>
                <option value="BIKE">自行车</option>
                <option value="E_BIKE">电动车</option>
            </select>
        </div>
    </div>

```

图 3-2-6



不同始末点检索方式的实现：

1. 根据接口通过调取数据库中 NavNode 表中的 name, 将不同地点的 name 在前端以下拉框的方式展示出来供用户选择。

```
@RestController
@RequestMapping("@"/api/test")
public class TestController {

    @Autowired
    private NavNodeMapper nodeMapper;

    @GetMapping("@"/nodes")
    public ResponseEntity<?> getNodes() {
        try {
            List<NavNode> nodes = nodeMapper.selectList( queryWrapper: null);
            return ResponseEntity.ok(nodes);
        } catch (Exception e) {
            e.printStackTrace();
            return ResponseEntity.status(500).body("Error retrieving nodes: " + e.getMessage());
        }
    }
}
```

图 3-2- 7

```
// 加载下拉框数据
fetch( input: "http://localhost:8080/api/nav/nodes") Promise<Response>
    .then(res : Response => res.json()) Promise<any>
    .then(nodes => {
        allNodes = nodes;
        const start : HTMLInputElement = document.getElementById( elementId: "start");
        const end : HTMLInputElement = document.getElementById( elementId: "end");

        nodes.forEach(n => {
            const o1 : HTMLOptionElement = new Option(n.name, n.id);
            const o2 : HTMLOptionElement = new Option(n.name, n.id);
            start.add(o1);
            end.add(o2);
        });

        saveOriginalOptions(start);
        saveOriginalOptions(end);
    });
```

图 3-2- 8

2. 根据接口通过调取数据库中 NavNode 表中的 name, 将不同地点的 name 在前端以下拉框的方式展示出来供用户选择：

使用 `filterSelectOptions` 函数为两个搜索输入框添加了 `input` 事件监听器，函数中传入的两个参数分别为要过滤的下拉框元素和搜索关键词。先检查下拉框是否有原始选项备份（`originalOptions`），如果没有备份，则从当前选项创建备份，清空当前下拉框的所有选项。

根据搜索词过滤原始选项，匹配包含搜索词的选项，将匹配的选项重新添加到下拉框，如果没有匹配项，显示“未找到匹配项”；使用 `option.text.toLowerCase().includes(searchTerm)` 进行模糊匹配，可匹配地点名称中的任意部分，在页面加载时，通过 `saveOriginalOptions` 函数保存原始选项，这样可以在每次搜索时基于原始数据进行过滤，而不是基于已过滤的数据。

```
// 实时搜索功能
const startSearchInput : HTMLInputElement = document.getElementById( 'elementId: "start-search"' );
const startSelect : HTMLInputElement = document.getElementById( 'elementId: "start"' );

startSearchInput.addEventListener( type: "input", listener: function() :void {
    const searchTerm : string = this.value.toLowerCase();
    filterSelectOptions(startSelect, searchTerm);
});

const endSearchInput : HTMLInputElement = document.getElementById( 'elementId: "end-search"' );
const endSelect : HTMLInputElement = document.getElementById( 'elementId: "end"' );

endSearchInput.addEventListener( type: "input", listener: function() :void {
    const searchTerm : string = this.value.toLowerCase();
    filterSelectOptions(endSelect, searchTerm);
});

// 过滤下拉选项
function filterSelectOptions(selectElement, searchTerm) :void {
    // 获取所有原始选项（从一个隐藏的备份中）
    if (!selectElement.originalOptions) {
        selectElement.originalOptions = [];
        for (let i : number = 0; i < selectElement.options.length; i++) {
            selectElement.originalOptions.push({
                value: selectElement.options[i].value,
                text: selectElement.options[i].text
            });
        }
    }
}
```

图 3-2-9

```
// 清空当前选项
selectElement.innerHTML = '';

// 根据搜索词过滤并添加选项
const filteredOptions :any[] = selectElement.originalOptions.filter(option =>
    option.text.toLowerCase().includes(searchTerm)
);

if (filteredOptions.length > 0) {
    filteredOptions.forEach(option => {
        const opt :HTMLOptionElement = new Option(option.text, option.value);
        selectElement.add(opt);
    });
} else {
    const opt :HTMLOptionElement = new Option( text: '未找到匹配项', value: '');
    selectElement.add(opt);
}
}
```

图 3-2- 10

## 展示沿途建筑简介的实现方法：

在校园导航系统中，沿途建筑简介功能的实现是通过后端 NavNode 实体类中的 description 字段存储建筑描述信息，当 Dijkstra 路径规划算法计算出路径后，这些包含描述信息的节点数据会通过 API 返回给前端，在 script.js 中通过遍历路径节点将建筑名称和描述信息显示在前端的结果面板的"途径路段及建筑介绍"部分。

```
function nav() :void { 显示用法
    const s = document.getElementById( elementId: "start").value;
    const e = document.getElementById( elementId: "end").value;
    const mode = document.getElementById( elementId: "mode").value;

    fetch( input: `http://localhost:8080/api/nav/path?start=${s}&end=${e}&mode=${mode}` ) Promise<Response>
        .then( res : Response => res.json() ) Promise<any>
        .then( res => {
            // 显示结果
            document.getElementById( elementId: "distance").textContent = res.distance.toFixed( fractionDigits: 1);
            document.getElementById( elementId: "time").textContent = (res.time / 60).toFixed( fractionDigits: 1);
            document.getElementById( elementId: "result").classList.add("show");

            // 显示路径详情
            const pathDetails :HTMLElement = document.getElementById( elementId: "path-details");
            if (pathDetails) {
                let pathHtml :string = '<h4>途径路段及建筑介绍</h4><ul class="path-list">';
                res.path.forEach((node, index) :void => {
                    pathHtml += '<li class="path-item">
                        <span class="path-index">${index + 1}</span>
                        <span class="path-name">${node.name}</span>
                        <span class="path-desc">${node.description || '暂无描述'}</span>
                    </li>';
                });
                pathHtml += '</ul>';
                pathDetails.innerHTML = pathHtml;
            }

            const data = res.path;
            const points = data.map(p => new BMap.Point(p.lng, p.lat));
```

图 3-2- 11

点击地图上建筑展示内容：

通过获取数据库中 NavNode 表中的对应地点的信息，如日常开门时间和假期开门时间进行对比判断对应的时间显示状态颜色（绿、黄、红）和显示时间（日常/假期）。

```
function formatTimeDisplay(timeStr):string { // 显示用法
  if (!timeStr || timeStr === '未知') {
    return '<span class="info-value">未知</span>';
  }

  // 检查是否为时间段格式
  const timeRangeMatch = timeStr.match(/(\d{1,2}:\d{2})-(\d{1,2}:\d{2})/);
  if (!timeRangeMatch) {
    return '<span class="info-value">' + timeStr + '</span>';
  }

  const [, openTime, closeTime] = timeRangeMatch;
  const now:Date = new Date();
  const currentTime:number = now.getHours() * 60 + now.getMinutes(); // 转换为分钟

  // 解析开门和关门时间
  const [openHour:number, openMin:number] = openTime.split(':').map(Number);
  const [closeHour:number, closeMin:number] = closeTime.split(':').map(Number);
  const openTimeInMinutes:number = openHour * 60 + openMin;
  const closeTimeInMinutes:number = closeHour * 60 + closeMin;

  // 计算距离关门时间的分钟数（30分钟内视为即将关闭）
  const minutesToClose:number = closeTimeInMinutes - currentTime;
  const isClosed:boolean = currentTime < openTimeInMinutes || currentTime > closeTimeInMinutes;
  const isClosingSoon:boolean = minutesToClose > 0 && minutesToClose <= 30; // 30分钟内关闭视为即将关闭
  const isOpen:boolean = !isClosed && !isClosingSoon;

  let statusText:string = '';
  let statusClass:string = '';
  let displayTime:string = openTime + '-' + closeTime;
```

图 3-2-12

```
// 查询该位置的建筑信息
fetch(input: 'http://localhost:8080/api/location/coordinates?lng=${point.lng}&lat=${point.lat}') Promise<Response>
  .then(res:Response => res.json()) Promise<any>
  .then(data => {
    if (data) {
      // 获取当前日期，判断是工作日还是周末
      const now:Date = new Date();
      const dayOfWeek:number = now.getDay(); // 0是周日，6是周六
      const isWeekend:boolean = (dayOfWeek === 0 || dayOfWeek === 6); // 周六日为周末

      // 根据是否为周末选择开放时间
      const openTimeLabel:string = isWeekend ? "假期开门时间" : "平时开门时间";
      const openTimeValue = isWeekend ? data.openTimeHoliday : data.openTimeSchool;

      // 创建信息窗口内容，使用函数格式化时间
      const content:string = `
        <div class="info-window">
          <h4>${data.name || '未知地点'}</h4>
          <div class="info-item">
            <span class="info-label">描述,</span>
            <span class="info-value">${data.description || '暂无描述'}</span>
          </div>
          <div class="info-item">
            <span class="info-label">${openTimeLabel},</span>
            <span class="info-value">${formatTimeDisplay( timeStr: openTimeValue || '未知')}</span>
          </div>
        </div>
      `;
    }
  });
```

图 3-2-13

```
@GetMapping(Ⓜ✓"/path")
public Map<String, Object> path(
    @RequestParam Long start,
    @RequestParam Long end,
    @RequestParam TravelMode mode) {

    List<NavNode> path = navService.findPath(start, end);

    double distance = navService.calcDistance(path);
    double time = distance / mode.getSpeed(); // 秒

    Map<String, Object> res = new HashMap<>();
    res.put("path", path);
    res.put("distance", distance);
    res.put("time", time);

    return res;
}
```

图 3-2- 14



### 3.3 实现过程（运行结果）

使用 Dijkstra 算法搜索最短路径，如图所示，图 3-1 在数据表（NavEdge）构建的图中未增加“青瓷实训基地”及左侧路段节点时，耗时 8 分钟，增加节点后图 3-2 中导航用时减少到 7.2 分钟。



图 3-3-1 路线规划效果图



图 3-3-2 路线规划效果图

支持两种方式确定起点和终点：使用下拉框或关键字检索通过下拉框获取始末地点。



图 3-3-3 下拉方式效果图



图 3-3-4 搜索方式效果图

支持对导航路段周边建筑介绍，不仅可以输出导航路线还可展示沿途的建筑和路段的情况。



图 3-3-5 路段介绍效果图

点击地图上的建筑可获取详情介绍，支持查询建筑的描述和开放时间。





图 3-3-6 实时介绍效果图

支持选择不同的出行方式, 根据不同的出行会产生“预计时间不同”的效果。

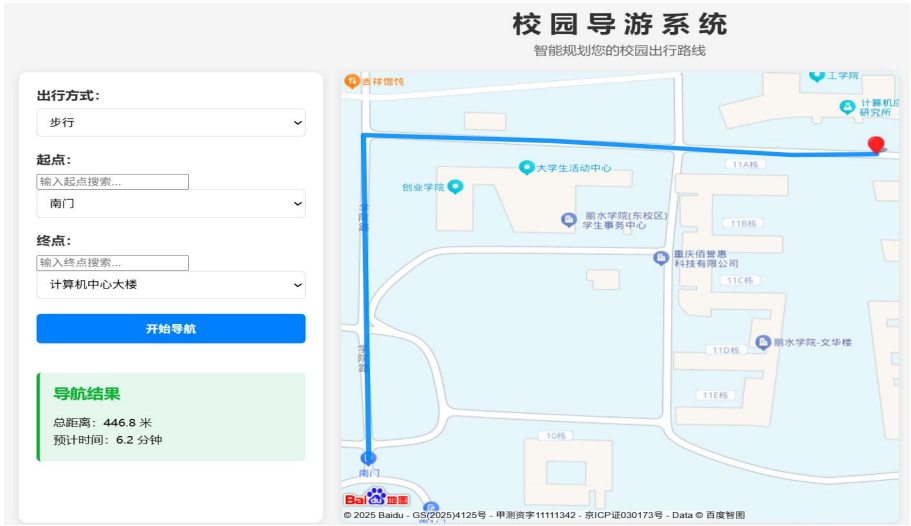


图 3-3-7 步行时间效果图

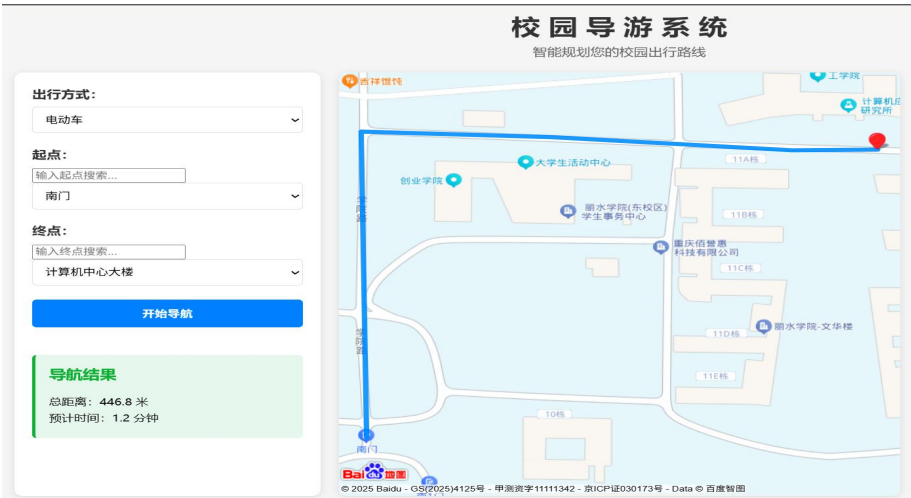


图 3-3-8 电动车时间效果图

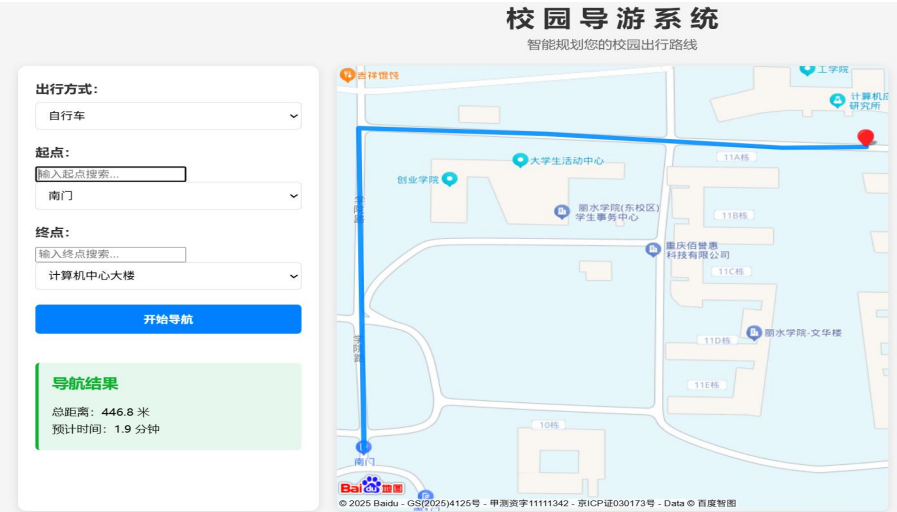


图 3-3-9 自行车时间效果图

针对不同时间，相应建筑的开放时间会根据工作日/假期和营业时间进行状态（绿、黄、红）的更改。

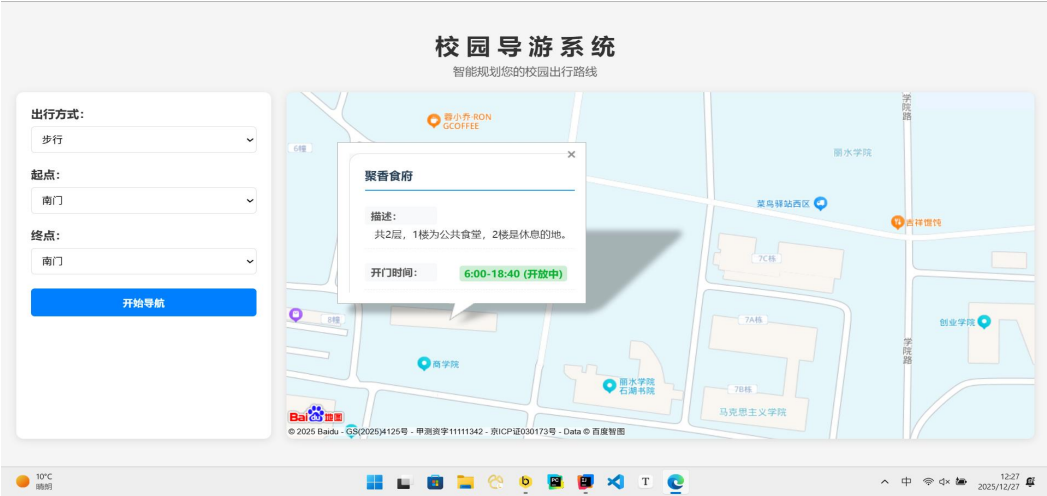


图 3-3-10 绿色状态效果图



图 3-3-11 黄色状态效果图



图 3-3-12 红色状态效果图